

Module 17 — Power BI connecté à SQL

Bootcamp Data Analyst — From Zero to Hero | Niveau Intermédiaire · Module 17

Ce que tu seras capable de faire à la fin de ce module

- Expliquer ce qu'est Power BI, à quoi il sert, et la différence Desktop/Service
- Te repérer dans l'interface (vue Rapport, volets Visualisations/Champs, canevas)
- Connecter Power BI à une vraie base PostgreSQL (celle du Module 16) en mode **Import**
- Construire un dashboard à partir d'une **vue SQL** plutôt que de tables brutes
- Écrire une mesure DAX simple
- Comprendre pourquoi le choix Import vs DirectQuery compte concrètement sur une base serverless

 **Durée estimée** : 2h30  **Outils** : Power BI Desktop (gratuit, Windows) · ta base Neon du Module 16  **Prérequis** : Module 16 terminé (la vue `vue_resume_clients` doit exister)

1. Qu'est-ce que Power BI, et à quoi ça sert

Power BI est un outil de **Business Intelligence (BI)** édité par Microsoft : il sert à transformer des données brutes (fichiers Excel, bases de données, APIs, fichiers CSV...) en **rapports visuels interactifs** — des dashboards que tu peux filtrer, explorer, et partager avec d'autres personnes.

Le problème qu'il résout

Imagine que tu envoies chaque semaine un fichier Excel avec un TCD à ta direction. Trois soucis reviennent vite :

- Le fichier est figé au moment où tu l'envoies — personne ne peut filtrer par lui-même sans te redemander une version.
- Si la donnée source change, il faut tout refaire à la main.
- Difficile de connecter plusieurs sources (une base SQL, un fichier Excel, une feuille Google) dans un seul rapport cohérent.

Power BI répond à ces trois problèmes : il se connecte **directement** aux sources de données (dont une base PostgreSQL, comme tu vas le faire ici), permet de **rafraîchir** le rapport en un clic quand la donnée change, et livre un dashboard que le destinataire peut **filtrer lui-même**, sans avoir besoin de toi pour changer de vue.

Desktop vs Service — deux briques distinctes

	POWER BI DESKTOP	POWER BI SERVICE
Quoi	Application à installer, pour construire un rapport	Plateforme en ligne (powerbi.com), pour publier et partager un rapport
Coût	Gratuit	Gratuit avec limites, payant pour le partage en entreprise (Pro/Premium)
Ce module	On l'utilise pour tout ce module	Mentionné en fin de module, pas obligatoire pour la suite

Dans ce module, on reste sur **Power BI Desktop** — l'outil de construction. Publier sur le Service est une étape ultérieure, une fois le rapport prêt.

Installer Power BI Desktop

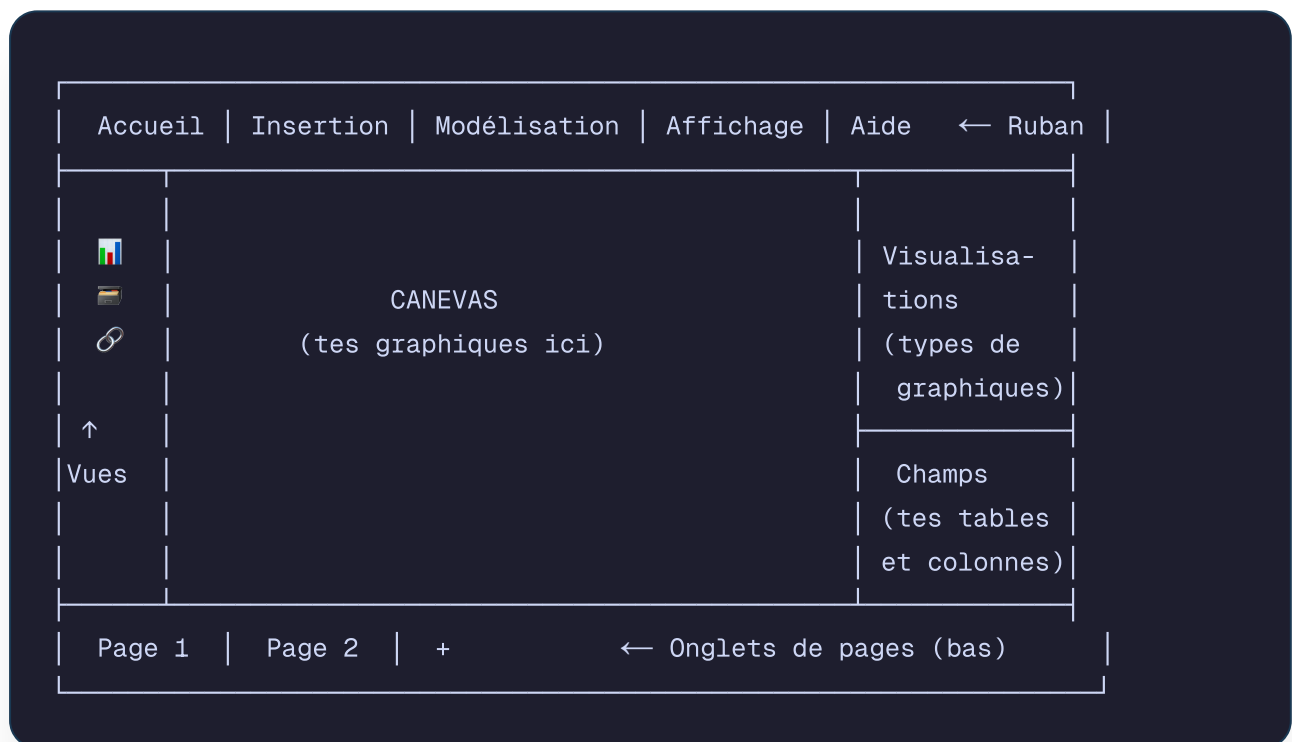
Power BI Desktop est **gratuit**, mais **Windows uniquement**.

- **Windows** : télécharge-le sur powerbi.microsoft.com/desktop ou via le Microsoft Store.

- **Mac** : Power BI Desktop ne fonctionne pas nativement. Trois options : une machine Windows (salle informatique, VM), ou Power BI Service directement dans le navigateur (fonctionnalités de connexion à une base externe plus limitées côté gratuit) — dans ce module, on part du principe que tu as accès à Power BI Desktop sous Windows.

2. Tour de l'interface

Au premier lancement, Power BI Desktop affiche une page d'accueil (raccourcis récents, connexions rapides) — ferme-la pour arriver sur l'espace de travail principal. Repère ces zones, tu vas les utiliser tout le long du module :



ZONE	À QUOI ELLE SERT
Ruban (haut)	Actions principales : obtenir des données, mettre en forme, gérer les relations entre tables
 Icônes de vues (bord gauche)	Bascule entre 3 vues : Rapport (tes dashboards), Données (tes tables, façon tableur), Modèle (les relations entre tables, façon schéma SQL)
Canevas (centre)	La zone où tu construis visuellement ton dashboard — tu y déposes des graphiques
Volet Visualisations (droite, haut)	La palette de types de graphiques disponibles (barres, courbes, cartes, tableaux...)
Volet Champs (droite, bas)	La liste de tes tables et colonnes — tu glisses une colonne d'ici vers le canevas ou vers un visuel
Onglets de pages (bas)	Un rapport Power BI peut avoir plusieurs pages, comme des feuilles dans un classeur Excel

 **Pont Excel → Power BI** : le **Volet Champs** correspond à la liste des colonnes que tu voyais en construisant un TCD Excel. Le **Canevas + Volet Visualisations** remplace le TCD lui-même — sauf qu'ici, plusieurs graphiques interagissent entre eux sur une même page, ce qu'Excel ne fait pas nativement.

C'est dans la **vue Rapport** que tu vas passer le plus clair de ton temps ce module — c'est la vue par défaut à l'ouverture.

3. Se connecter à ta base Neon en mode Import

Récupérer ta chaîne de connexion — la bonne version

Dans la console Neon, clique sur **Connect**. Tu vas voir deux versions de la chaîne de connexion : une avec **-pooler** dans le nom d'hôte, une sans. **Utilise la version SANS -pooler (connexion directe)** pour Power BI.

⚠ **Pourquoi pas la version poolée ?** La connexion poolée (PgBouncer) est optimisée pour plein de petites requêtes rapides et successives — pas pour une requête d'import unique qui peut prendre du temps à rapatrier toutes les données. Un import Power BI sur une connexion poolée a plus de risques d'être coupé en cours de route. La connexion directe est plus adaptée à ce cas précis.

Se connecter

1. Dans Power BI Desktop : `Accueil → Obtenir les données → Base de données → Base de données PostgreSQL`.
2. Un premier écran te demande deux champs : **Serveur** (ex. `ep-cool-darkness-123456.us-east-2.aws.neon.tech`) et **Base de données** (ex. `neondb`).
3. Choisis le mode **Import** (pas DirectQuery — voir pourquoi ci-dessous).
4. Un second écran te demande de t'authentifier : choisis **Base de données**, saisis ton nom d'utilisateur et ton mot de passe Neon.

👉 Pourquoi Import et pas DirectQuery ?

- **Import** : Power BI copie les données une fois en mémoire. Ensuite, tout est instantané, même hors connexion — il faut juste penser à rafraîchir de temps en temps.
- **DirectQuery** : chaque interaction avec un visuel envoie une requête live à la base. Sur une base serverless comme Neon qui se met en veille après 5 minutes d'inactivité (Module 16), ça veut dire qu'un simple clic sur un filtre peut déclencher un réveil de la base et une latence perceptible — voire une erreur si la base était en train de se suspendre pile à ce moment.

Import est donc le choix le plus fiable ici. C'est aussi cohérent avec le volume de données (une base d'exercice, pas des millions de lignes qui saturent la mémoire).

4. Se connecter à la vue, pas aux tables brutes

Dans la fenêtre de navigation qui s'affiche, tu vois toutes les tables **et vues** de ta base :

`clients`, `transactions`, et `vue_resume_clients` (créée au Module 16).

Coche `vue_resume_clients` plutôt que de reconstruire la logique de jointure/agrégation dans Power BI. C'est exactement l'intérêt d'avoir préparé cette vue en SQL : Power BI la traite comme une simple table, sans avoir besoin de connaître la logique de jointure

`clients` / `transactions` qu'elle contient.

Clique sur **Charger**. Power BI importe les données — patiente quelques secondes si ta base venait de se réveiller.

5. Construire un premier dashboard

Dans le volet **Champs** à droite, tu retrouves les colonnes de `vue_resume_clients` : `nom`, `ville`, `date_inscription`, `nb_transactions`, `total_transactions`.

Construis au minimum :

1. **Une carte (Card)** : glisse `total_transactions` dedans, change l'agrégation en **Somme** — ça affiche le total général.
2. **Un graphique en barres** : `ville` en axe, `total_transactions` en valeur (Somme) — classement des villes par montant total.
3. **Un tableau** : `nom`, `ville`, `nb_transactions`, `total_transactions` — le détail client par client, trié par `total_transactions` décroissant.

 **Pont Excel → Power BI** : ce que tu fais ici (glisser une colonne en Lignes, une autre en Valeurs) est exactement la logique d'un TCD Excel — Power BI ajoute juste l'interactivité entre les visuels (clique sur une barre du graphique, le tableau se filtre automatiquement).

6. Une première mesure DAX

Une **mesure** est un calcul qui se recalcule automatiquement selon les filtres actifs sur le dashboard — contrairement à une colonne calculée qui se fige à l'import.

Dans le volet Champs, clic droit sur `vue_resume_clients` → **Nouvelle mesure** :

DAX

```
Montant moyen par client = AVERAGE(vue_resume_clients[total_transactions])
```

Ajoute cette mesure dans une nouvelle carte. Si tu filtres ensuite ton dashboard sur une seule ville (clique sur une barre du graphique), cette carte recalcule automatiquement la moyenne **pour cette ville uniquement** — c'est ce recalcul dynamique qui distingue une mesure d'une simple colonne.

👉 Une deuxième mesure à essayer

DAX

```
Part des transactions supérieures à 20000 =  
DIVIDE(  
    CALCULATE(COUNTROWS(vue_resume_clients),  
        vue_resume_clients[total_transactions] > 20000),  
    COUNTROWS(vue_resume_clients)  
)
```

CALCULATE modifie le contexte de filtre pour ne compter que les clients au-dessus du seuil ;

DIVIDE évite une erreur de division par zéro (contrairement à un simple `/`).

✅ Résumé du module

CONCEPT	CE QU'IL FAUT RETENIR
Power BI	Outil de BI (Microsoft) : transforme des données en rapports visuels interactifs, connectés à leur source
Desktop vs Service	Desktop = construire (gratuit, Windows) ; Service = publier/partager (powerbi.com)
Vue Rapport / Données / Modèle	Les 3 vues principales : construire le dashboard / voir les tables façon tableur / gérer les relations entre tables
Import vs DirectQuery	Import copie les données une fois (fiable, rapide) ; DirectQuery interroge en direct (risqué sur une base qui scale-to-zero)
Connexion directe vs poolée	Utiliser la chaîne sans <code>-pooler</code> pour un import fiable sur Neon
Se connecter à une vue	Une vue SQL se comporte comme une table pour Power BI — prépare la logique complexe en amont, en SQL
Carte / Graphique en barres / Tableau	Les visuels de base, équivalents à un TCD Excel
Mesure DAX	Calcul qui se recalcule selon les filtres actifs — <code>AVERAGE</code> , <code>CALCULATE</code> , <code>DIVIDE</code>

Quiz — Vérifie ta compréhension

Q1. Pourquoi privilégier le mode **Import** plutôt que **DirectQuery** pour se connecter à une base Neon ?

- a) DirectQuery n'existe pas pour PostgreSQL
- b) Neon met sa base en veille après inactivité — DirectQuery interroge en direct à chaque interaction, ce qui est plus fragile sur ce type de base
- c) Import est le seul mode gratuit

👉 Voir la réponse

✓ b) — DirectQuery envoie une requête à la base à chaque clic sur un visuel. Sur une base serverless qui peut être suspendue, ça introduit de la latence, voire des erreurs. Importer les données une fois, ce qui est plus stable pour ce contexte.

Q2. Pourquoi utiliser la chaîne de connexion sans `-pooler` pour l'import Power BI ?

- a) La version poolée n'existe pas sur le tier gratuit
- b) La connexion poolée est optimisée pour de nombreuses petites requêtes rapides, pas pour une requête d'import unique potentiellement longue
- c) `-pooler` est réservé aux connexions Python/R

👉 Voir la réponse

✓ b) — Le pooler (PgBouncer en mode transaction) est pensé pour un flux de courtes requêtes indépendantes. Un import Power BI est une requête unique qui peut prendre du temps — la connexion directe est plus fiable pour ce cas.

Q3. Pourquoi se connecter à `vue_resume_clients` plutôt qu'aux tables `clients` et `transactions` séparément ?

- a) Power BI ne sait pas faire de jointures
- b) La vue centralise la logique de jointure/agrégation en SQL — Power BI la consomme comme une simple table, sans avoir à la recréer
- c) Les vues se rafraîchissent plus vite que les tables

👉 Voir la réponse

✓ b) — C'est tout l'intérêt des vues vues au Module 16 : préparer une fois la logique complexe (jointures, agrégations) en SQL, puis la réutiliser telle quelle dans n'importe quel outil qui sait lire une base PostgreSQL.

Pour aller plus loin

Ce module couvre les bases pour te connecter et construire un premier dashboard — Power BI est un outil vaste (visuels avancés, DAX poussé, mise en forme). Pour approfondir en autonomie, la chaîne YouTube [Power BI University](#) propose des tutoriels supplémentaires sur l'outil.

Module suivant

Tu as un vrai dashboard connecté à une vraie base. Dans le **Module 18**, on passe aux statistiques inférentielles — interpréter une p-value et un test d'hypothèse.

→ **Module 18 — Statistiques inférentielles**